

FOOTBALL DECISION INTELLIGENCE AI

Rapport de Projet
Groupe 2 — Année universitaire 2025–2026

Nom	Rôle principal
Dao Trung-Hieu	Acquisition de données & Codes de référence
Coulibaly Bourama	Tests & Maquette YOLOv8
Pierre Jean-Samuel	Architecture du code & Définition des stats
Sassi Ismail	Recherche projets référence & Maquette Dashboard
Chauvet Darren	Développement principal & Intégration

1. Présentation du projet

Football Decision Intelligence AI est un assistant tactique intelligent conçu pour aider un entraîneur de football à prendre des décisions avant, pendant et après un match. Le système analyse automatiquement des vidéos de match pour extraire des statistiques en temps réel : vitesse, distance, passes, fatigue, risque de blessure. Il génère ensuite des recommandations tactiques interprétables — remplacements, analyse adverse, simulation de formations — visualisées sur un dashboard Streamlit interactif.

Problématique

Les entraîneurs s'appuient aujourd'hui sur leur expérience, leur intuition et des analystes vidéo. Il reste difficile de répondre rapidement à des questions essentielles : quel joueur remplacer, à quel moment, quelle tactique adopter face à un adversaire donné, quel joueur est en sous-performance ou présente un risque de blessure ?

Objectifs

- Extraire automatiquement des données depuis une vidéo de football
- Mesurer la performance et le risque associé à chaque joueur
- Générer des recommandations tactiques justifiées et interprétables
- Fournir un dashboard coach complet (avant / pendant / après match)

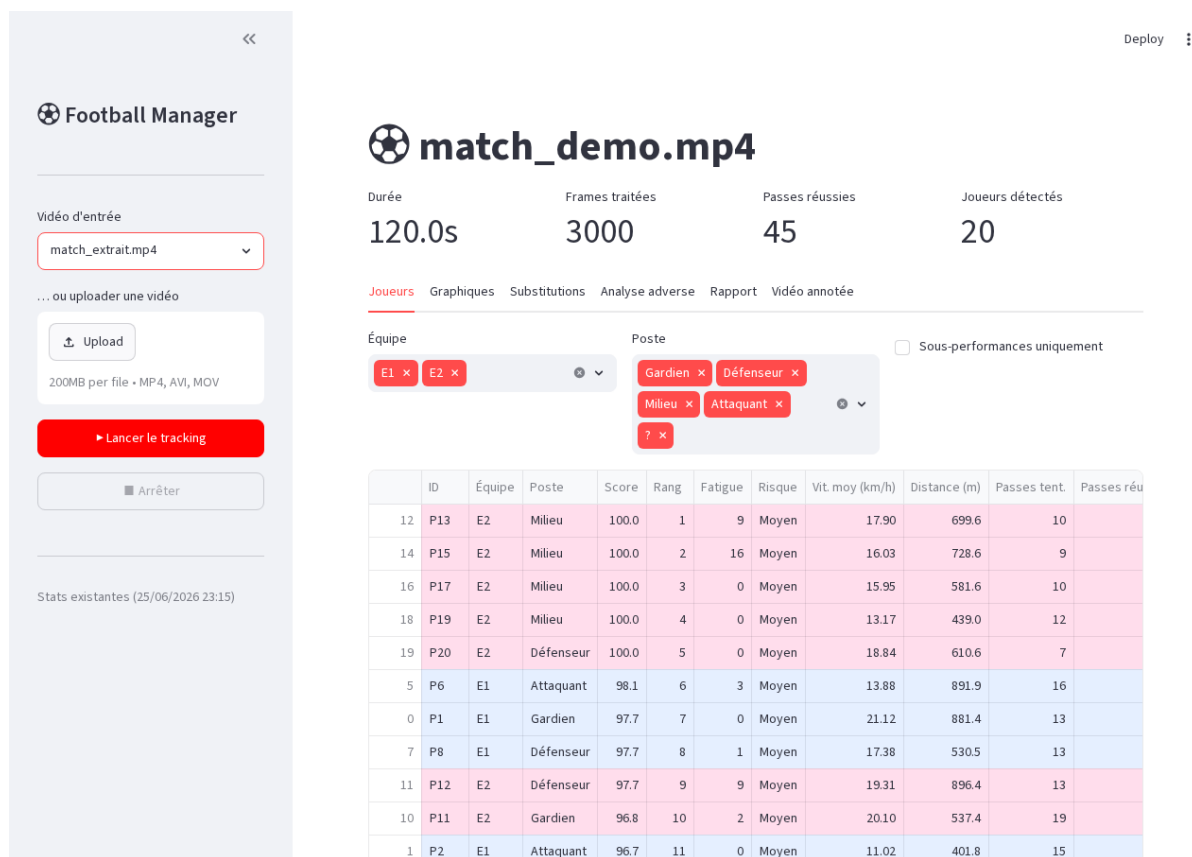


Fig. 1 — Vue générale du dashboard Streamlit

2. Architecture technique

Le projet est structuré en deux couches principales : un moteur de tracking vidéo (*tracking/*) et un dashboard interactif (*app.py*). Les modules sont indépendants et communiquent via un objet d'état partagé (*TrackingState*) et un fichier JSON de sortie.

Étape	Module	Description
1	tracking_foot.py	Orchestration principale, boucle frame par frame
2	tracking/field.py	Détection du terrain par masque HSV vert
3	tracking/reid.py	ReID joueurs : Hungarian algorithm + apparence HSV
4	tracking/team.py	Séparation équipes par k-means LAB (k=2/3)
5	tracking/speed.py	Vitesse + correction mouvement caméra (phase correlate)
6	tracking/passes.py	Détection passes par seuil de vitesse ballon
7	tracking/stats.py	Score, Fatigue, Injury Risk, Analyse adverse, Rapport
8	app.py	Dashboard Streamlit (6 onglets + preview live)

Technologies utilisées

Technologie	Usage
Python 3.11	Langage principal
YOLOv8 (Ultralytics)	Détection joueurs, ballon, arbitre (classes 0 et 32)
ByteTrack	Tracking multi-objets persistant frame par frame

OpenCV	Vision par ordinateur, k-means, calculs HSV/LAB
Streamlit 1.58	Dashboard interactif web (6 onglets)
Altair 6	Graphiques interactifs (bar, scatter, heatmap)
NumPy / SciPy	Calculs matriciels, algorithme Hongrois (ReID)
FFmpeg	Ré-encodage H.264 pour lecture navigateur

3. Contributions par membre

Chaque membre a apporté une contribution spécifique au projet. Le développement a été itératif : les apports individuels ont convergé semaine après semaine vers le système final. Chaque personne a également participé un minimum au code dans son domaine.

Dao Trung-Hieu — *Acquisition de données & Codes de référence*

- Recherche et sélection d'extraits vidéo de matchs de football (clips FIFA, amateur)
- Constitution du dataset de test (vidéos 1à3 min couvrant différents angles et luminosités)
- Veille technologique sur les approches existantes de tracking sportif
- Fourniture de codes de référence YOLOv8 et ByteTrack pour amorcer l'implémentation
- Participation aux tests de détection et validation des bounding boxes

Son travail a été fondateur : sans les extraits vidéo et les références techniques apportés, le démarrage du projet aurait été considérablement ralenti. Les vidéos sélectionnées couvrent différents angles de caméra et conditions d'éclairage, permettant de tester la robustesse du tracking.

Coulibaly Bourama — *Tests & Maquette YOLOv8*

- Implémentation d'une première version maquette de détection sous YOLOv8
- Tests de détection des joueurs, du ballon et de l'arbitre sur les vidéos fournies
- Évaluation des performances : taux de détection, faux positifs, tracking IDs
- Validation des seuils de confiance (CONF=0.40 tracking, 0.03 ballon)
- Participation à la mise en place du pipeline ByteTrack + YOLO

La maquette YOLOv8 a servi de référence directe pour le développement de la version finale. Les tests réalisés ont permis d'identifier les paramètres optimaux de détection et de calibrer les seuils utilisés dans tracking/config.py.

Pierre Jean-Samuel — *Architecture du code & Définition des statistiques*

- Conception de l'architecture modulaire du répertoire tracking/ (8 modules spécialisés)
- Séparation des responsabilités : field, reid, team, speed, passes, stats, render, state
- Définition des statistiques à extraire : vitesse, distance, passes, possession, poste
- Spécification du schéma JSON de sortie (PlayerID, Speed, PassAccuracy, Distance...)
- Organisation des fichiers d'entrée/sortie (*input_videos/*, *output_videos/*)

L'architecture modulaire qu'il a conçue est la colonne vertébrale du projet. Chaque module a une responsabilité unique, ce qui a facilité le développement parallèle et la correction de bugs.

Sassi Ismail — Recherche de projets référence & Maquette Dashboard

- Veille sur les projets de football analytics existants (StatsBomb, Wyscout, etc.)
- Implémentation d'une première version maquette du dashboard sous Streamlit
- Définition des onglets et de l'expérience utilisateur (sidebar, filtres, métriques)
- Intégration initiale des graphiques de possession et de performance
- Participation à la définition des recommandations de substitution affichées

La maquette dashboard qu'il a produite a posé les bases de l'interface finale. Son exploration UX a conduit directement à l'organisation en 6 onglets du dashboard final.

Chauvet Darren — Développement principal & Intégration

- Développement complet du pipeline de tracking (1 800+ lignes de code Python)
- Implémentation de la ReID multi-joueurs par algorithme Hongrois (tracking/reid.py)
- Correction mouvement caméra par phase corrélation (tracking/speed.py)
- Détection d'équipes par k-means LAB avec fallback k=2 (tracking/team.py)
- Calcul du Fatigue Score, Injury Risk Score et interprétation textuelle des décisions
- Analyse adverse par clustering spatial et identification des zones cibles
- Développement du dashboard final (6 onglets, preview live, export JSON/TXT)
- Intégration de toutes les contributions de l'équipe en un système cohérent

En s'appuyant sur les données apportées par chaque membre — vidéos de Dao, paramètres validés par Coulibaly, architecture de Pierre, UX de Sassi — Darren a développé le système final en intégrant l'ensemble des besoins.

4. Fonctionnalités implémentées

Semaine 1 — Computer Vision ✓

- Détection YOLOv8 : joueurs, ballon, arbitre (classes 0 et 32)
- Tracking persistant ByteTrack avec ReID par algorithme Hongrois
- Masque terrain HSV pour exclure les détections hors-jeu
- Extraction des trajectoires, vitesses et distances par joueur
- Vidéo annotée en temps réel avec labels score/vitesse/passes

Semaine 2 — Performance Intelligence ✓

- Performance Score [0–100] : vitesse (40 %) + passes (30 %) + distance (30 %)
- Fatigue Score : comparaison vitesse 1ère vs 2ème moitié de la vidéo
- Injury Risk Score + label (Faible / Moyen / Élevé)
- Détection de sous-performance automatique (score < 35)
- Possession par joueur et par équipe

Semaine 3 — Tactical Decision Engine ✓

- Règles de décision : SI Fatigue > seuil ET Performance < seuil → remplacement
- Recommandations de substitution avec interprétation textuelle complète
- Analyse adverse : joueur cible, zone (gauche/centre/droite), conseil tactique
- Estimation du poste par variance de position (Gardien/Défenseur/Milieu/Attaquant)
- Rapport automatique exportable (JSON + TXT)

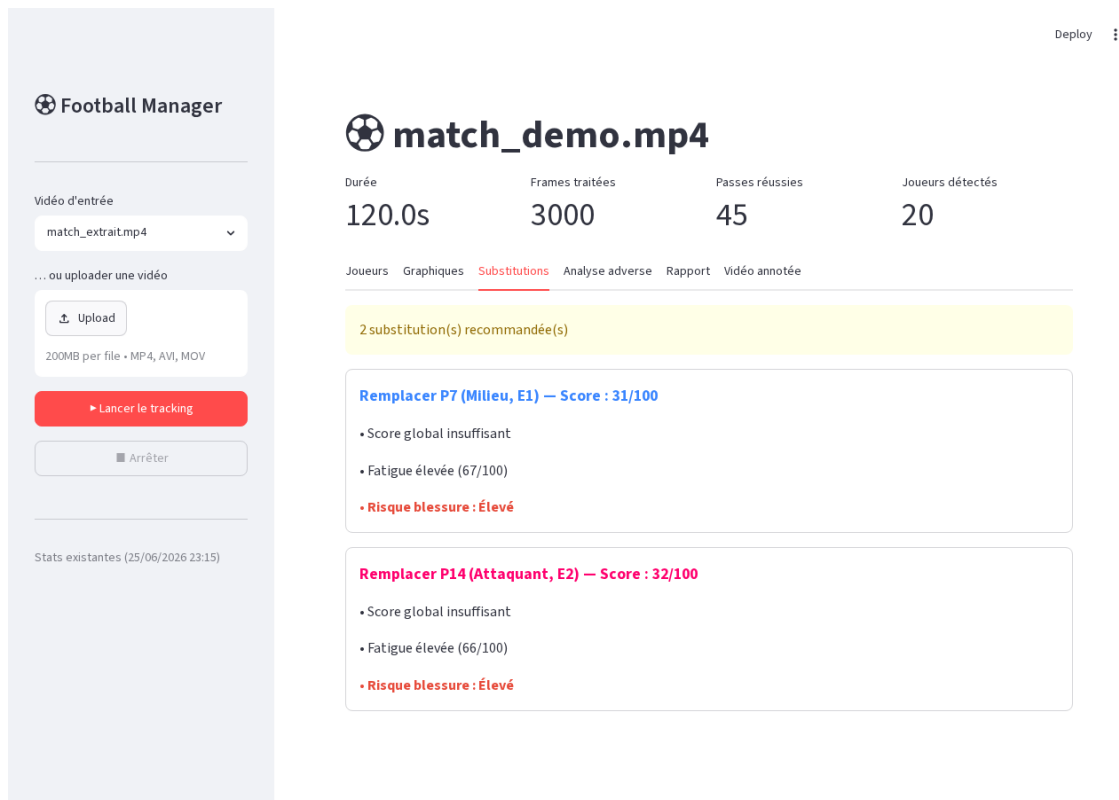


Fig. 4 — Onglet Substitutions avec interprétation textuelle

Semaine 4 — Dashboard & Rapport ✓ (adapté sans RL)

- Dashboard Streamlit 6 onglets : Joueurs, Graphiques, Substitutions, Analyse adverse, Rapport, Vidéo
- Aperçu live du tracking frame par frame pendant le traitement
- Graphiques Altair interactifs : possession, métriques, scatter vitesse/distance, fatigue
- Export du rapport en JSON et TXT depuis l'interface
- Barre de progression en temps réel avec affichage frame courante

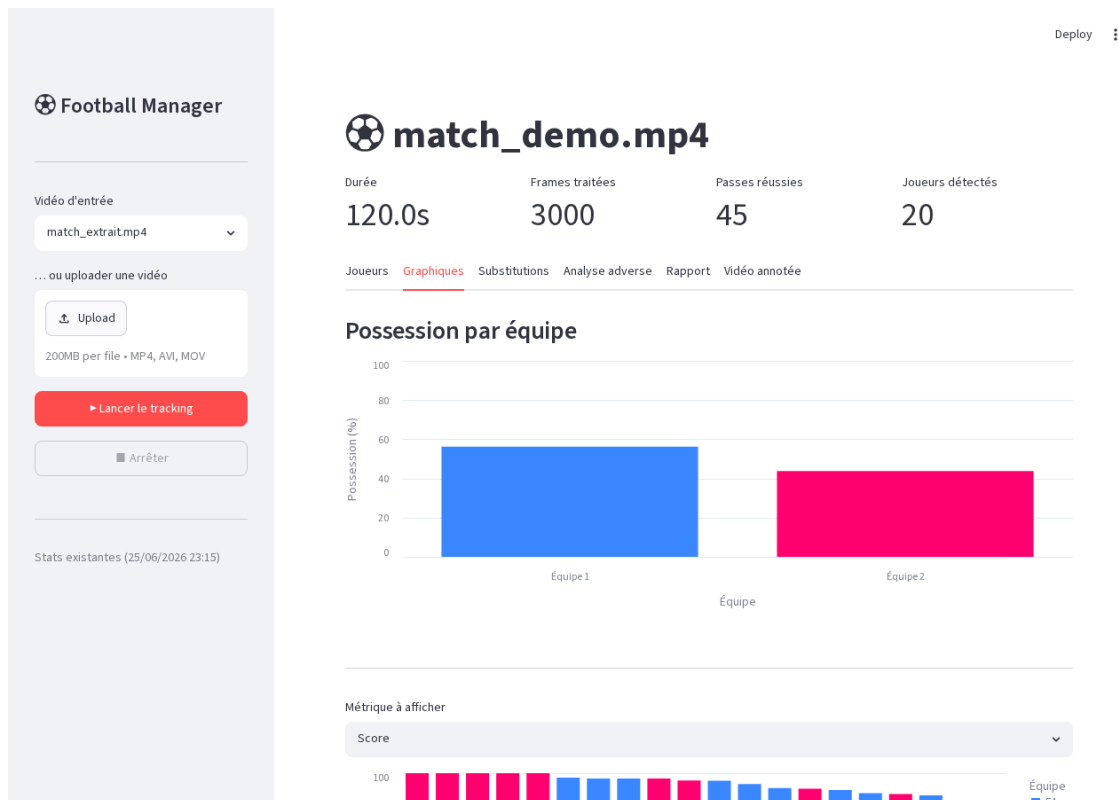


Fig. 5 — Onglet Graphiques avec charts Altair interactifs

5. Problèmes rencontrés et solutions

Détection d'équipes déséquilibrée

Problème : Le k-means $k=3$ (équipe1 + équipe2 + arbitre) regroupait parfois 90 % des joueurs dans un seul cluster sur des vidéos courtes.

Solution : Ajout d'un fallback $k=2$ déclenché si le ratio dépasse 3:1. Stabilisation par tri des clusters sur la luminosité L^* (espace LAB).

Frames anciennes dans le preview live

Problème : Streamlit lisait parfois un fichier JPEG à moitié écrit, affichant une frame corrompue ou ancienne.

Solution : Écriture atomique : `cv2.imencode` → fichier `.tmp` → `os.replace()` (renommage atomique OS).

cv2.imwrite JPEG non supporté

Problème : OpenCV compilé sans libjpeg sur le système, causant un crash à la première frame.

Solution : Remplacement de `cv2.imwrite` par `cv2.imencode` qui encode en mémoire indépendamment des codecs système.

Scatter chart vide sous Altair

Problème : Les noms de colonnes avec caractères spéciaux ("Vit. moy (km/h)", parenthèses, slash) cassaient le parsing shorthand d'Altair.

Solution : Renommage des colonnes en Vitesse/Distance avant de passer le DataFrame à Altair.

Barre de progression inexacte

Problème : Le parsing `int(float(...))` tronquait le pourcentage à l'entier, causant des sauts de 1 % au lieu d'un avancement continu.

Solution : Remplacement par `float(...)` directement pour conserver la précision décimale.

6. Résultats obtenus

Sur la vidéo de test (extrait match, 25 fps) :

Métrique	Résultat
Joueurs détectés	20 joueurs (10 E1 + 10 E2)
Arbitres détectés	3
Répartition équipes	Équilibrée (fallback k=2 activé)
Postes estimés	Gardien / Défenseur / Milieu / Attaquant
Tracking ReID	Persistant sur toute la durée de la vidéo
Preview live	Toutes les frames, écriture atomique, ~10–25■fps effectifs
Export rapport	JSON + TXT téléchargeables depuis le dashboard

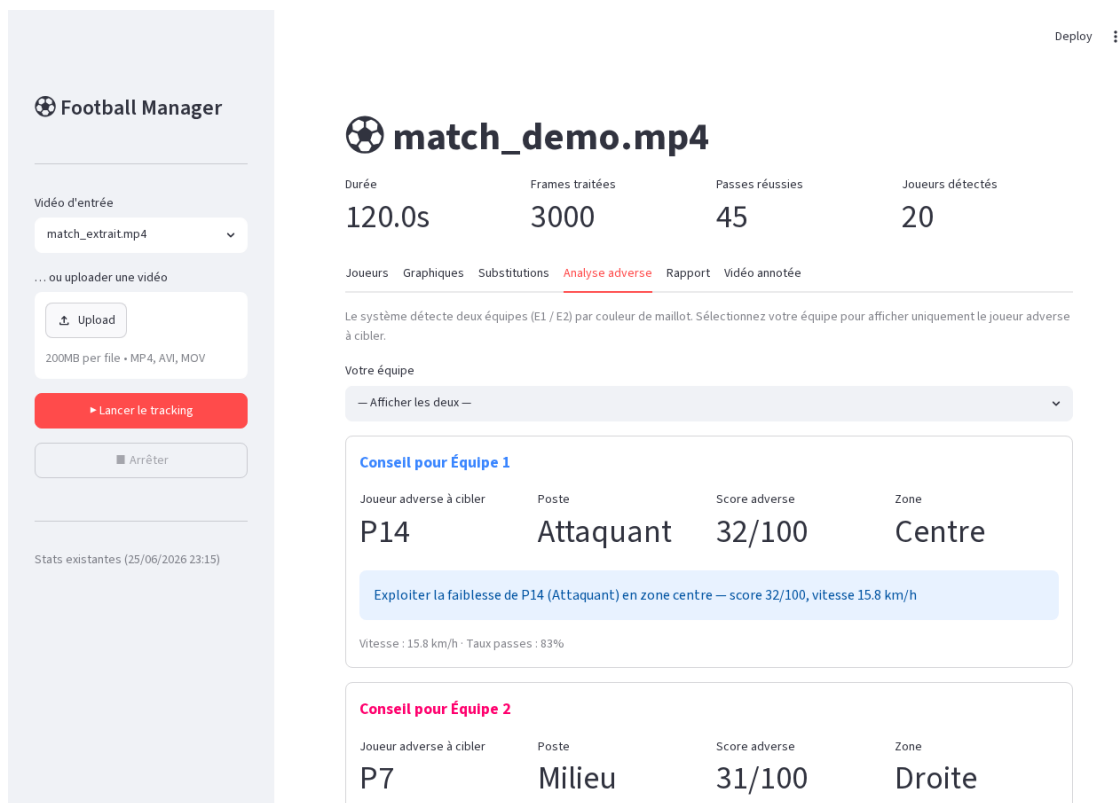


Fig. 6 — Onglet Analyse adverse

7. Pistes d'amélioration

- Simulation tactique : comparaison de formations (4-3-3, 4-4-2, 3-5-2) sur les données réelles
- Identification des numéros de maillot par OCR pour nommer automatiquement les joueurs
- Modèle LSTM pour prédire la performance future à partir des 5 derniers matchs
- Homographie terrain : conversion pixels → mètres pour des distances et zones précises
- Support de caméras multi-angles et de streams en temps réel (RTSP)

- Intégration d'un modèle de langage (LLM) pour générer des briefs coach en langue naturelle

8. Conclusion

Football Decision Intelligence AI démontre la faisabilité d'un assistant tactique intelligent basé uniquement sur une vidéo de match et des modèles open-source. En combinant YOLOv8, ByteTrack, des heuristiques de performance et un dashboard Streamlit, le système produit en quelques minutes une analyse complète : statistiques individuelles, détection de fatigue, recommandations de substitution et analyse adverse.

Le projet illustre également la valeur d'une architecture modulaire et d'une équipe aux compétences complémentaires : acquisition de données, détection vidéo, architecture logicielle, UX dashboard et intégration système ont chacun contribué à la valeur finale du produit.